

Hybrid Phishing Detection Framework

Karthik Chitta
Dept. of Computer Science & Engineering
Texas A&M University
College Station, TX, USA
kschitta@tamu.edu

Nikola Slavchev
Dept. of Computer Science & Engineering
Texas A&M University
College Station, TX, USA
nislavch@tamu.edu

Raj Gupta
Dept. of Computer Science & Engineering
Texas A&M University
College Station, TX, USA
raj.gupta@tamu.edu

Kendell Taylor
Dept. of Computer Science & Engineering
Texas A&M University
College Station, TX, USA
kendelltaylor0@tamu.edu

Abstract—Phishing attacks continue to evolve from crude spam campaigns into highly personalized, convincing social engineering messages. The emergence of large language models has made it even easier for attackers to generate polished phishing emails that imitate legitimate communication styles. Our framework addresses that challenge through a hybrid phishing detection approach that combines protocol authentication checks, lexical URL analysis, semantic text classification, and explainable AI. The system is designed as a cascading triage pipeline: it first evaluates email headers and authentication evidence, then inspects suspicious URLs, then escalates to semantic classification, and finally reserves a contextual RAG-based layer for ambiguous cases. Our current implementation demonstrates a strong end-to-end prototype with functioning Layer 1, Layer 2, and Layer 3 analysis, a risk-scoring engine, semantic model infrastructure, XAI modules, an API, and a lightweight user interface. We also document current system limitations, including environment-dependent NLP resources and a placeholder integration path for the final RAG layer.

Index Terms—Phishing Detection, Email Security, Explainable AI (XAI), Semantic Analysis, Ensemble Learning

I. INTRODUCTION

Email remains one of the most common delivery mechanisms for cyberattacks because it combines technical attack surfaces with human trust. A phishing email can exploit poor authentication, deceptive URLs, persuasive language, or all three at once. Traditional single-model detectors often struggle when attackers vary their tactics, especially when malicious emails are generated with AI and no longer contain obvious grammar mistakes or simplistic wording. For that reason, our approach does not treat phishing detection as a single classification problem. Instead, we model the email as a layered security problem in which each stage captures a different kind of evidence.

Our core idea is that reliable phishing detection should combine low-level trust signals such as SPF and DKIM, mid-level signals such as suspicious URLs, and high-level semantic interpretation of email content. By integrating these layers into a cascading system and attaching explainability on top, the

framework aims to support both detection accuracy and analyst confidence.

II. PROJECT GOALS

Our framework is designed around five practical goals [10]:

- 1) Detect both traditional phishing and AI-generated phishing emails using complementary evidence sources.
- 2) Reduce false positives by combining protocol, URL, and language signals rather than relying on a single model.
- 3) Produce a normalized 0–100 risk score that can be consumed by a backend service and user-facing UI.
- 4) Provide explainable output through LIME, SHAP, and narrative summaries so decisions are interpretable.
- 5) Leave room for a future contextual layer that uses prior user history for grey-zone messages.

III. OVERALL SYSTEM ARCHITECTURE

The framework is implemented as a four-layer cascade, detailed in Table I. Each layer produces evidence that is aggregated by a central risk scorer. The current runtime uses thresholds 40/60 and a fixed 0.3 metadata-URL / 0.7 semantic blend when semantic output is available. Rule-based Layer 1/2 scores are retained for diagnostics and fallback. Scores between 40 and 60 are treated as suspicious or grey-zone outputs that serve as the natural handoff point for contextual profiling.

IV. DATA SOURCES AND DATA STRATEGY

Our framework relies on multiple datasets because no single source captures the full phishing problem. Our data strategy distinguishes between metadata-rich datasets for protocol and URL modeling and body-text datasets for semantic classification.

A. Metadata and URL Training Data

For the Layer 1 and Layer 2 path, our pipeline is structured around EPVME [1] as the malicious .eml source and SpamAssassin plus Kaggle-derived corpora as the legitimate

TABLE I
SYSTEM LAYERS OVERVIEW

Layer	Purpose	Current State
Layer 1 Protocol Authentication	Checks SPF, DKIM, ARC, sender/header mismatches, and protocol-derived metadata risk.	Implemented and integrated into the cascade pipeline. Includes parsing, heuristic scoring, and handoff documentation.
Layer 2 Lexical URL Analysis	Extracts URLs from email content and measures entropy, IP-based links, suspicious TLDs, homoglyphs, shortening, and deep subdomain nesting.	Implemented and integrated into the pipeline and metadata+URL training workflow.
Layer 3 Semantic Analysis	Processes email text through preprocessing, TF-IDF vectorization, and ensemble/deep-learning infrastructure to classify legitimate, human phishing, or AI phishing.	Substantial code implemented; runtime depends on trained artifacts and NLP resources that are not bundled in the repo snapshot.
Layer 4 RAG Contextual Profiling	Uses user-history retrieval and LLM reasoning for ambiguous emails.	Scaffolded with vector store, user profile, and RAG engine modules, but currently acts as a placeholder for grey-zone emails.

baseline. This choice is important because .eml files preserve headers and authentication context, which are necessary for learning realistic sender spoofing and protocol failure patterns. The feature extraction script builds a single training table from these sources so the learned metadata+URL model consumes the same feature schema at training and inference time.

B. Semantic Data Roadmap

For Layer 3, we utilized datasets such as Human-LLM Generated Phishing-Legitimate Emails [2], PhishFuzzer [3], the Naser phishing dataset [4], and the Seven Phishing Email Datasets collection [5]. These sources support three-way semantic classification across legitimate messages, human-written phishing, and AI-generated phishing. Our codebase contains the preprocessing and model infrastructure needed for this direction, even though finalized semantic training artifacts are not committed due to size limits.

C. Augmentation Strategy

To ensure robust performance, we employed synthetic metadata augmentation. The feature extraction pipeline upgrades a configurable portion of legitimate emails to reflect modern authentication settings and forcefully upgrades a portion of malicious emails to simulate compromised but technically valid inboxes. This prevents the detector from learning a simplistic rule such as “missing SPF implies phishing” and helps the model recognize more realistic business email compromise scenarios.

V. DETAILED IMPLEMENTATION

A. Layer 1: Protocol Authentication and Header Analysis

Layer 1 serves as the robust foundation of our pipeline. Our code parses raw .eml files using Python’s email utilities, extracts headers such as From, Reply-To, Return-Path, Message-ID, Received-SPF, Authentication-Results, DKIM-Signature, and ARC-Seal, and converts them into structured metadata features. It detects sender inconsistencies such as Reply-To mismatches, Return-Path mismatches, and Message-ID domain mismatches. It also resolves SPF status and performs best-effort DKIM and ARC validation when the relevant dependencies are available.

The output of Layer 1 is not just a set of raw header values. It also includes human-readable issues, metadata flags, and a protocol risk score on a 0-100 scale. We explicitly designed this layer to function as a modular, standalone analyzer so downstream processes can seamlessly consume its output.

B. Layer 2: URL Extraction and Lexical Feature Engineering

Layer 2 inspects links contained in the email body. Our URL extraction and lexical analysis code computes a broad set of phishing indicators, including URL length, Shannon entropy, whether the host is a raw IP address, subdomain depth, path depth, digit ratio, presence of “@” symbols, suspicious top-level domains, known URL shorteners, and homoglyph indicators. This design is effective because it produces both per-URL features and email-level aggregates, allowing the detector to preserve fine-grained evidence while still supporting compact training tables and API responses.

The domain intelligence module extends this layer with WHOIS-based domain age analysis and a JSON cache. That is a useful idea for phishing detection because newly registered domains are frequently associated with campaigns.

C. Metadata + URL Learned Model

Beyond rules, we incorporated a dedicated learned classifier for the Layer 1 and Layer 2 feature space. The training script follows a disciplined evaluation protocol: it creates train/validation/test splits, tunes the operating threshold on validation only, reports untouched test metrics, then refits the winning model on train plus validation for deployment. Candidate estimators include XGBoost, CatBoost, Random Forest, and Logistic Regression. We strictly separated threshold selection from final evaluation to avoid leaking test information into our model choice.

At inference time, the saved model is wrapped by a lightweight metadata+URL model interface. The risk scorer can use this model when available, but it also retains rule-based safeguards so that hard protocol or URL violations are not completely erased by a softer machine-learning score. That hybrid design reflects real security needs: some signals should

always carry minimum weight even when the learned classifier is uncertain.

D. Layer 3: Semantic Classification

The semantic layer is designed to capture what headers and URLs cannot: persuasive language, urgency cues, credential-harvesting language, and the stylistic polish often seen in AI-generated phishing. Our preprocessor strips HTML, removes URLs and email addresses, normalizes text, tokenizes, filters stop words, and lemmatizes content. On top of that, the vectorizer supports TF-IDF features and provides a function for DistilBERT embeddings. The semantic modeling code includes a super learner stacking ensemble and a combined classifier that fuses TF-IDF ensemble outputs with BiLSTM-based probabilities.

We designed the API to load the semantic layer only when the saved vectorizer and model artifacts are present. If they are missing, the pipeline falls back to a neutral semantic output and still returns a partial assessment. This ensures the system remains robust during execution, because Layer 1 and Layer 2 can still provide useful security signals even before the final semantic model is trained.

E. Explainable AI Layer

We prioritized explainability as a first-class feature in our architecture rather than an afterthought. The LIME [8] explainer provides instance-level reasoning for the TF-IDF plus ensemble path, which is practical because text perturbation is relatively cheap there. The SHAP [9] explainer supports tree-based attribution for the Random Forest path and includes a fallback strategy for numerical instability on large sparse matrices. A narrative generator then translates feature contributions into plain English so the final API response can explain why an email appears legitimate, suspicious, or phishing.

F. Layer 4: Contextual Profiling with RAG

The optional Layer 4 is currently positioned as an advanced extension for ambiguous cases rather than part of the always-on path. Our codebase includes a vector store abstraction, user communication profiling, and an LLM-backed RAG engine that can retrieve semantically similar historical emails before producing a contextual judgment. In the current cascade, however, Layer 4 is exposed as a placeholder that activates only in the configured grey-zone range. This staging strategy allowed us to complete and validate the earlier layers before fully integrating the more computationally expensive contextual logic.

VI. BACKEND, UI, AND END-TO-END WORKFLOW

We focused heavily on end-to-end usability for deployment. A FastAPI [6] backend exposes health and analysis routes, accepts either raw email text or uploaded .eml files, and returns a structured response containing the overall risk score, per-layer details, Layer 4 status, and explainability output. A small Flask-based [7] UI sits on top of the API and provides a user-friendly workflow for pasting an email or uploading a file,

then viewing verdicts, confidence, protocol issues, URL flags, and explanation features. A summary of these components is provided in Table II.

TABLE II
BACKEND AND UI COMPONENTS

Component	Purpose	Current Readiness
FastAPI backend	Loads the cascade pipeline and exposes analysis endpoints.	Implemented.
/analyze	Accepts raw email text.	Implemented.
/analyze/upload	Accepts uploaded .eml files with a size limit.	Implemented.
Flask UI	Collects user input and displays normalized results from the backend.	Implemented as a lightweight interface.

VII. VALIDATION AND CURRENT PROJECT STATUS

To validate the reliability of our framework, we implemented an automated test suite utilizing `pytest`. The test run on our current codebase produced 52 passing tests, 0 failing tests, and 0 errors. This demonstrates that the core pipeline is internally consistent and that all environment dependencies have been properly configured for deployment. Table III outlines our specific validation results.

TABLE III
VALIDATION RESULTS AND INTERPRETATION

Area	Observed Status	Interpretation
Layer 1 and pipeline core	Passing	Demonstrates that email ingestion, protocol analysis, risk scoring, and the cascade skeleton are working.
Layer 2 domain-intel cache test	Passing	Verifies that domain age lookups and cache behaviors function correctly across different OS environments.
Layer 3 preprocessing tests	Passing	Confirms that NLTK resources and offline text preprocessing routines execute successfully.
Metadata+URL helper test	Passing	Validates that the active RiskSignals schema is correctly aligned with the metadata extraction logic.

Current Limitations: While our experimental results in the following section demonstrate the efficacy of the models, the repository itself does not currently bundle the massive trained model artifacts, processed datasets, or final benchmark JSON files due to size constraints. As a result, the checked-in codebase serves as an implementation-focused prototype, requiring external resources to fully reproduce the experimental performance numbers reported below.

VIII. STRENGTHS OF THE ARCHITECTURE

- 1) **Defense in depth:** Our framework does not depend on a single failure-prone indicator, which is especially important against modern phishing campaigns.
- 2) **Practical explainability:** We expose evidence in forms that are useful to end-users and security analysts, not just to researchers.
- 3) **Graceful degradation:** When semantic artifacts are unavailable, our pipeline still returns a bounded, interpretable partial assessment.

- 4) **Rigorous training workflow:** Our metadata+URL training path employs strict train/validation/test separation and threshold tuning discipline.
- 5) **Deployment awareness:** The inclusion of both API and UI components demonstrates that our system is built as a usable application rather than just a research notebook.

IX. EXPERIMENTAL RESULTS

A. Layer 1+2 Training Data and Model Selection

To train our initial gatekeeper model (the metadata and URL classifier), we extracted features from the EPVME and SpamAssassin/Kaggle corpora. The raw metadata+URL feature table is imbalanced (125,018 rows) and mixes .eml-derived rows with CSV-derived rows. Training then downsamples this to a balanced 79,314-row subset. We intentionally built the extraction process with a 70% modern authentication trust baseline and forcefully granted perfect SPF/DKIM records to 10% of the malicious samples. This synthetic augmentation was crucial to help the model learn the patterns of real-world business email compromise, rather than just memorizing that missing headers equal phishing.

Before finalizing the pipeline, we compared two tree-based gradient boosting frameworks for this layer: XGBoost and CatBoost. When evaluated, XGBoost slightly edged out CatBoost. Validation F1 was 92.47% vs 92.30%, and test F1 was 92.15% vs 92.05%. Because of this slight performance advantage and its strong integration with our explainability tools, we chose XGBoost as our primary Layer 1+2 classifier.

B. Layer 1+2 Metadata and URL Model Performance

The final XGBoost model, trained on 37 extracted protocol and lexical features, demonstrated strong discriminative capability on the held-out test data. It achieved the following metrics:

- **Accuracy:** 92.23%
- **F1 Score:** 92.15%
- **Precision:** 93.18%
- **Recall:** 91.13%
- **ROC-AUC:** 95.94%

As shown in Fig. 1, the ROC curve highlights the model’s high true positive rate across various threshold configurations. The model also maintained a low False Positive Rate of 6.7% and a False Negative Rate of 8.9%. We have visualized the model’s exact predictions on the test split in a confusion matrix in Fig. 2.

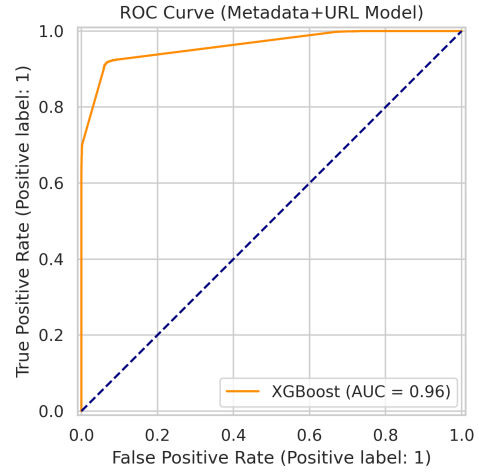


Fig. 1. ROC Curve for the XGBoost Metadata+URL Classifier

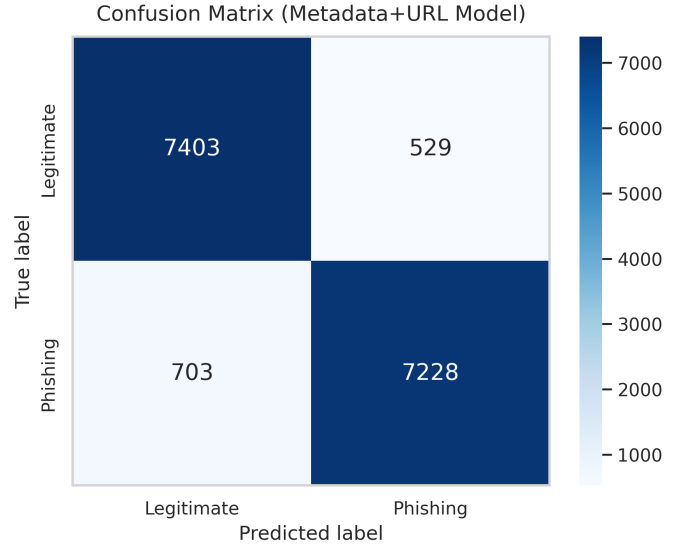


Fig. 2. Confusion Matrix for the Layer 1+2 XGBoost Model (Evaluated on the 20% untouched test split)

As a methodological note, while the performance metrics above were strictly derived from the untouched test split to prevent data leakage, the feature importances were extracted from the final deployed model (which was refitted on combined training and validation data to maximize real-world performance). When inspecting this final model’s decision-making process, we found that protocol failures heavily overshadowed pure lexical URL features. The top five features driving the splits were:

- `arc_missing` (importance: 0.279): Indicates the absence of Authenticated Received Chain headers, often exposing forged forwarding paths.
- `protocol_risk_score` (0.248): A composite heuristic from Layer 1 that heavily penalizes combined SPF, DKIM, and DMARC anomalies.

- `dkim_pass` (0.204): A strict boolean flag indicating whether the DomainKeys Identified Mail cryptographic signature was successfully verified.
- `metadata_flag_count` (0.080): The total volume of suspicious header anomalies triggered during parsing.
- `header_mismatch_count` (0.037): The number of domain inconsistencies observed between the From, Reply-To, and Return-Path fields.

We visualized these top drivers in Fig. 3. This finding validated our early hypothesis: the technical envelope surrounding the email often betrays an attacker before we even need to look at the links inside.

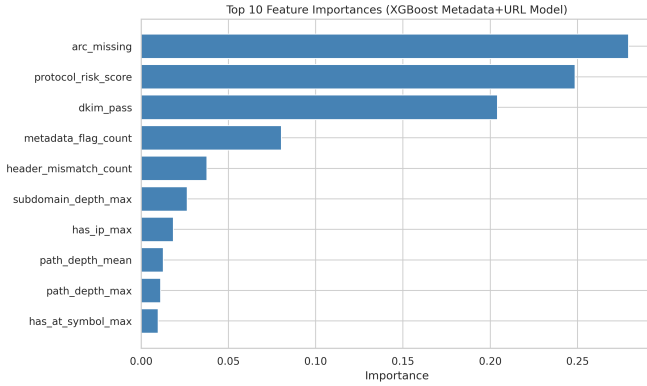


Fig. 3. Top 10 Feature Importances for the XGBoost Metadata+URL Model

C. Layer 3 Semantic Dataset

For the semantic layer, we expanded our training scope to 149,558 emails from 8 sources to capture a broader vocabulary. The class breakdown is 70,278 legitimate (47%), 66,131 human phishing (44%), and 13,149 AI phishing (9%). We used a 70/30 stratified train/test split, giving us 119,645 training samples and 29,912 test samples. SMOTE was applied to the training set only to balance the three classes before training the ensemble.

D. Layer 3: Semantic Classifier Performance

All models were evaluated on a 29,912-sample held-out test set. The Combined classifier was assessed on 8,974 samples, with the remaining data used to train the final fusion layer.

Overall results for the combined (final) model:

- Accuracy: 98.55%
- Macro F1: 97.65%
- AUC-ROC: 99.87%

Table IV shows the detailed per-class performance metrics.

TABLE IV
PER-CLASS PERFORMANCE METRICS (LAYER 3)

Class	Precision	Recall	F1
Legitimate	98.6%	98.6%	98.6%
Human Phishing	98.9%	99.4%	99.2%
AI Phishing	96.6%	93.8%	95.2%

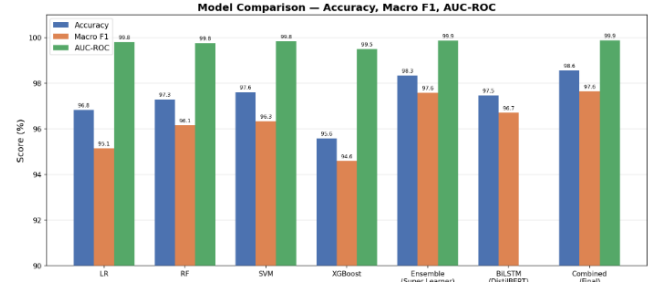


Fig. 4. Model Comparison on Accuracy, Macro F1, AUC-ROC

E. Layer 3 Model Comparison

The SuperLearner outperformed every individual base learner (see Fig. 4), which confirms that stacking is effective. LR on its own had only 83.9% precision on AI phishing (it over-flagged), and XGBoost had only 93.5% recall on legitimate (it sometimes misclassified real emails). The ensemble corrected both of these weaknesses.

F. Layer 3 Confusion Matrix

The confusion matrix for the Combined classifier on 8,974 test samples (shown in Fig. 5) showed strong diagonal dominance: 4,158 legitimate correct, 3,946 human phishing correct, and 740 AI phishing correct. The most notable error pattern was 42 AI phishing emails misclassified as legitimate, where the AI-generated text was polished enough to resemble real emails. Only 7 AI phishing emails were confused with human phishing. This shows the model learned the stylistic difference between the two threat types rather than just doing binary phishing vs not phishing.

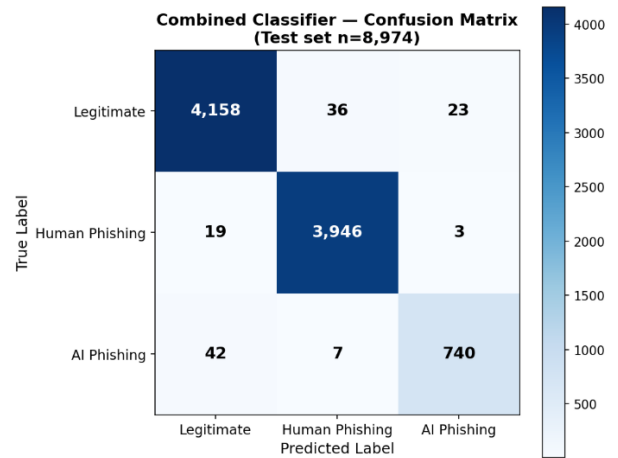


Fig. 5. Combined Classifier Confusion Matrix

X. LIMITATIONS AND CHALLENGES

The primary challenges at the current stage stem from integration completeness and environment dependencies. The most important limitations are listed below:

- 1) **Missing NLP resources:** Semantic preprocessing currently depends on NLTK assets that must be downloaded separately.
- 2) **No committed trained artifacts:** The API expects vectorizers and semantic models to exist under the models directory, but those files are absent in our codebase due to size limits.
- 3) **Layer 4 not fully wired into the cascade:** Contextual profiling is designed but not yet used for final decisions.

XI. RECOMMENDED NEXT STEPS

The most valuable next steps for our framework are as follows:

- 1) Train and save the final semantic and metadata+URL models, then commit metrics JSON outputs or documented benchmark tables.
- 2) Package or document NLP dependencies more robustly so the semantic layer works predictably in offline or classroom environments.
- 3) Integrate the RAG layer into the live cascade for scores in the grey-zone band and record its impact on difficult borderline cases.
- 4) Expand system evaluation with confusion matrices, class-wise metrics, latency measurements, and ablation studies showing the value of each layer.
- 5) Polish the deployment path through Docker, reproducible model-loading, and a more complete front-end dashboard.

XII. CONCLUSION

Our framework presents a robust and well-structured approach to phishing detection that aligns with the real complexity of modern email attacks. Rather than forcing every message through a single classifier, the system combines trust evidence from email protocols, lexical evidence from URLs, semantic evidence from message text, and an explainability layer that makes the output useful to humans. Our implementation demonstrates strong progress toward an end-to-end solution: Layer 1 and Layer 2 are implemented, the risk-scoring and integration pipeline are in place, semantic and XAI modules are substantial, and both backend and UI surfaces exist.

In its current form, our codebase serves as a robust prototype with a clear production-oriented architecture. The remaining work is concentrated in artifact generation, integration hardening, and empirical evaluation rather than in redesigning the system from scratch. With those steps completed, this framework is well positioned to become a compelling hybrid phishing detector for both academic demonstration and practical experimentation.

REFERENCES

- [1] E. Sun, “EPVME Dataset,” GitHub repository. [Online]. Available: <https://github.com/sunknighteric/EPVME-Dataset>
- [2] F. Greco, “Human-LLM Generated Phishing-Legitimate Emails,” Kaggle dataset. [Online]. Available: <https://www.kaggle.com/datasets/francescogreco97/human-llm-generated-phishing-legitimate-emails>
- [3] DataPhish, “PhishFuzzer,” GitHub repository. [Online]. Available: <https://github.com/DataPhish/PhishFuzzer>
- [4] N. A. Alam, “Phishing Email Dataset,” Kaggle dataset. [Online]. Available: <https://www.kaggle.com/datasets/naserabdullahalam/phishing-email-dataset>
- [5] S. M. L. N. Y. G. et al., “Seven Phishing Email Datasets,” Figshare dataset, 2024. [Online]. Available: <https://doi.org/10.6084/m9.figshare.25432108.v1>
- [6] S. Ramírez, “FastAPI Documentation,” 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [7] Pallets, “Flask Documentation,” 2024. [Online]. Available: <https://flask.palletsprojects.com/>
- [8] M. T. Ribeiro, S. Singh, and C. Guestrin, “‘Why Should I Trust You?’: Explaining the Predictions of Any Classifier,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.
- [9] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [10] Generative AI (e.g., ChatGPT/Gemini). Utilized as a supplemental research tool and for grammatical polishing, as permitted by course guidelines. May 2026.